

Java Backend Architecture

Interview Series

Chapter 9.7

HTTP Caching

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.7 – HTTP Caching

Introduction

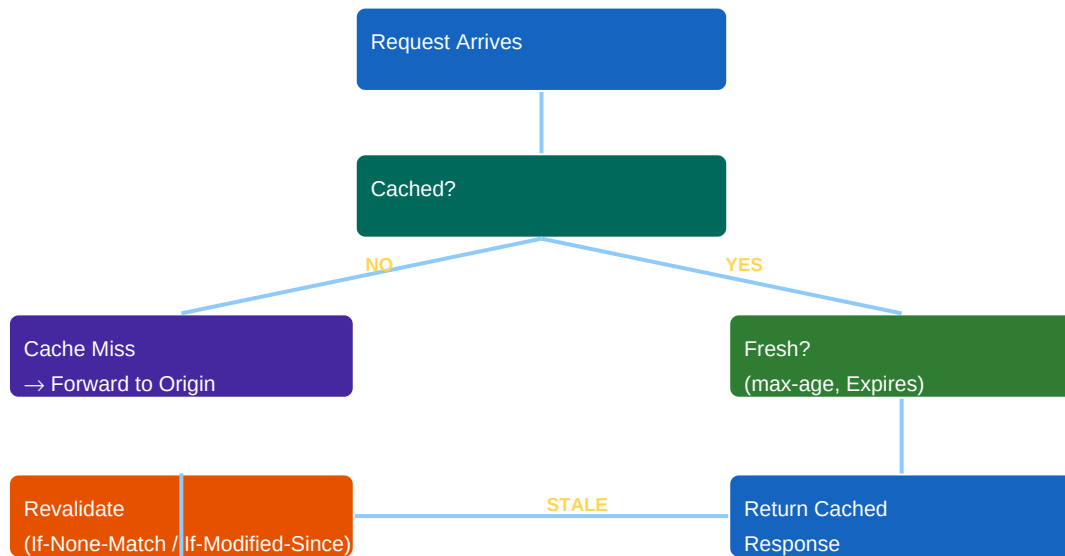
HTTP caching is defined in RFC 9111 and leverages the HTTP protocol itself to cache responses in browsers, CDNs, reverse proxies, and API gateways — without any application code changes. By correctly setting **Cache-Control**, **ETag**, **Last-Modified**, and **Vary** headers, Java backends can dramatically reduce origin server load and improve client-perceived latency. Spring MVC and Spring WebFlux provide first-class support via `CacheControl` builder, `ShallowEtagHeaderFilter`, and `ResponseEntity` headers.

Core Concepts & Definitions

Cache-Control	HTTP header controlling caching behavior for responses; replaces Expires header.
max-age	Number of seconds a response is considered fresh after it is served.
no-cache	Response may be cached but must be revalidated with origin server before reuse.
no-store	Response must never be stored in any cache (sensitive data: passwords, PII).
must-revalidate	Once stale, cache must not serve the response — must go to origin.
s-maxage	Overrides max-age for shared caches (CDNs, reverse proxies) only.
stale-while-revalidate	Serve stale response while asynchronously revalidating in background.
immutable	Asset will never change; do not revalidate even after max-age expires (for versioned assets).
ETag	Entity tag: opaque token representing a specific version of a resource. Strong or weak.
Last-Modified	Date/time of last resource modification; used for conditional GET validation.
If-None-Match	Client sends its cached ETag; server returns 304 if ETag matches.
If-Modified-Since	Client sends Last-Modified date; server returns 304 if not modified since.
304 Not Modified	Response indicating cached copy is still valid; no response body transmitted.
Vary	List of request headers that affect response content; cache keyed on Vary headers.
Surrogate-Key / Cache-Tag	CDN-specific header for batch purging related cache entries by tag.
Conditional GET	GET request with If-None-Match / If-Modified-Since; allows validation without full transfer.

HTTP Cache Decision Flow

HTTP Cache Decision Flow (RFC 9111)



no-store: never cache; no-cache: cache but always revalidate; must-revalidate: no stale use after expire

Cache-Control Directives Reference

Cache-Control Directive Map

max-age=N Cache for N seconds Relative to request time	no-cache Cache but revalidate every request (ETag)	no-store Never cache (sensitive data)
must-revalidate No stale use after expiry — go to origin	private Only browser cache not shared/CDN	public Cacheable by shared caches and CDN
s-maxage=N Max age for shared caches (overrides max-age)	stale-while-revalidate Serve stale, refresh async in background	immutable Never revalidate (versioned assets)

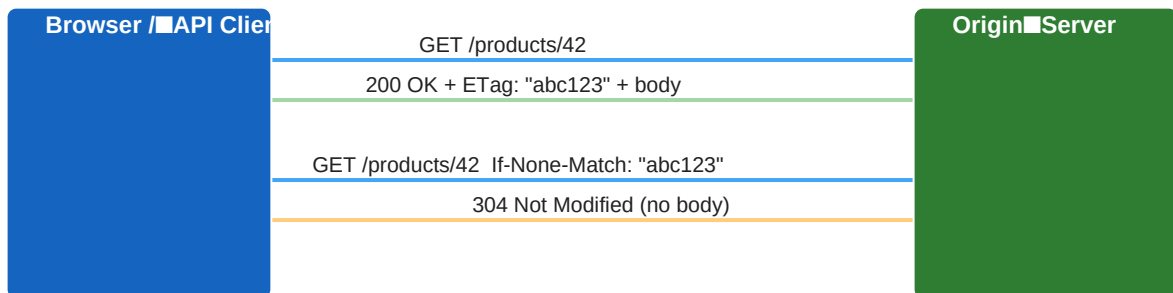
Spring MVC: Setting Cache-Control Headers

```
// ResponseEntity with Cache-Control @GetMapping("/products/{id}") public
ResponseEntity<Product> getProduct(@PathVariable Long id) { Product product =
productService.findById(id); CacheControl cc = CacheControl .maxAge(Duration.ofMinutes(30))
.cachePublic() // CDN-cacheable .mustRevalidate(); // stale not allowed return
ResponseEntity.ok() .cacheControl(cc) .body(product); } // No-cache for
authenticated/personalized responses @GetMapping("/user/profile") public
ResponseEntity<UserProfile> getProfile() { return ResponseEntity.ok()
.cacheControl(CacheControl.noCache().cachePrivate()) .body(profileService.getCurrent()); } //
No-store for sensitive data @GetMapping("/payments/{id}") public ResponseEntity<Payment>
getPayment(@PathVariable Long id) { return ResponseEntity.ok() }
```

```
.cacheControl(CacheControl.noStore()) .body(paymentService.findById(id)); }
```

ETag Implementation in Spring

ETag Conditional GET Flow



Strong ETag: "abc123" — byte-identical comparison

Weak ETag: W/"abc123" — semantically equivalent

304 saves bandwidth (no body); client reuses cached copy

Option 1: Shallow ETag Filter (automatic, content-based):

```
// Register ShallowEtagHeaderFilter — computes MD5 of response body @Bean public
FilterRegistrationBean<ShallowEtagHeaderFilter> shallowEtagFilter() {
    FilterRegistrationBean<ShallowEtagHeaderFilter> bean = new FilterRegistrationBean<>(new
    ShallowEtagHeaderFilter()); bean.addUrlPatterns("/api/*"); return bean; } //
ShallowEtagHeaderFilter: // 1. Buffer response body // 2. Compute MD5 → ETag:
"d41d8cd98f00b204e9800998ecf8427e" // 3. Compare with If-None-Match; return 304 if equal //
Caveat: still executes full business logic — only saves bandwidth, not server CPU
```

Option 2: Deep ETag (version-based, saves CPU):

```
@GetMapping("/products/{id}") public ResponseEntity<Product> getProduct( @PathVariable Long
id, WebRequest webRequest) { Product product = productService.findById(id); String etag = "\""
+ product.getVersion() + "\""; // checkNotModified() returns 304 if client ETag matches — exits
early if (webRequest.checkNotModified(etag)) { return null; // Spring returns 304, method not
continued } return ResponseEntity.ok() .eTag(etag)
.cacheControl(CacheControl.maxAge(Duration.ofMinutes(10))) .body(product); }
```

Last-Modified Conditional GET

```
@GetMapping("/articles/{id}") public ResponseEntity<Article> getArticle( @PathVariable Long
id, WebRequest webRequest) { Article article = articleService.findById(id); long lastModified =
article.getUpdatedAt().toEpochMilli(); // Returns 304 if client's If-Modified-Since >=
lastModified if (webRequest.checkNotModified(lastModified)) { return null; } return
ResponseEntity.ok() .lastModified(article.getUpdatedAt())
.cacheControl(CacheControl.maxAge(Duration.ofHours(1))) .body(article); } // Combined ETag +
Last-Modified (preferred) if (webRequest.checkNotModified(etag, lastModified)) { return null;
}
```

Vary Header

```
// Vary: Accept-Encoding — separate cache entry per encoding // Vary: Accept-Language —
separate per language // Vary: Accept — separate per content type (JSON vs XML) // Vary:
Authorization — NEVER (defeats caching for private content) @GetMapping("/reports/{id}") public
ResponseEntity<byte[]> getReport( @PathVariable Long id, @RequestHeader("Accept") String
```

```
accept) { byte[] content = reportService.generate(id, accept); return ResponseEntity.ok()
    .header(HttpHeaders.VARY, HttpHeaders.ACCEPT) .header(HttpHeaders.CONTENT_TYPE, accept)
    .cacheControl(CacheControl.maxAge(Duration.ofHours(1))) .body(content); }
```

Static Asset Caching: Immutable + Content Hashing

```
// Spring MVC Resource Handling – versioned static assets @Configuration @EnableWebMvc public
class WebMvcConfig implements WebMvcConfigurer { @Override public void
addResourceHandlers(ResourceHandlerRegistry registry) {
    registry.addResourceHandler("/static/**") .addResourceLocations("classpath:/static/")
    .setCacheControl( CacheControl.maxAge(Duration.ofDays(365)).immutable()) .resourceChain(true)
    .addResolver(new VersionResourceResolver() .addContentVersionStrategy("/**")); // MD5 hash in
    URL } } // Results in URLs like: /static/app-8f4d2a9c.js // Cache-Control: max-age=31536000,
    immutable // Fingerprinted URL changes when content changes → cache-busting
```

Reverse Proxy Cache (Nginx / Varnish)

```
# Nginx proxy cache config proxy_cache_path /var/cache/nginx levels=1:2
keys_zone=api_cache:10m max_size=1g inactive=60m use_temp_path=off; server { location /api/ {
    proxy_pass http://backend; proxy_cache api_cache; proxy_cache_key "$scheme$host$request_uri";
    proxy_cache_valid 200 10m; proxy_cache_valid 404 1m; proxy_cache_use_stale error timeout
    updating http_500 http_502 http_503; add_header X-Cache-Status $upstream_cache_status; } }
```

Spring WebFlux (Reactive) HTTP Caching

```
// Spring WebFlux – same CacheControl API @GetMapping("/products/{id}") public
Mono<ResponseEntity<Product>> getProduct( @PathVariable Long id, ServerWebExchange exchange) {
    return productService.findById(id) .map(product -> { String etag = "\"" + product.getVersion()
    + "\""; if (exchange.checkNotModified(etag)) { return
    ResponseEntity.status(HttpStatus.NOT_MODIFIED) .<Product>build(); } return ResponseEntity.ok()
    .eTag(etag) .cacheControl(CacheControl.maxAge(Duration.ofMinutes(30))) .body(product); }); }
```

Interview Questions & Answers

Q1. What is HTTP caching and which RFC defines it?

HTTP caching is the storage of HTTP responses by browsers, CDNs, and proxies for reuse in subsequent requests. RFC 9111 (HTTP Caching, 2022, updates RFC 7234) defines the caching model including freshness, validation, and directive semantics.

Q2. Explain the difference between Cache-Control: no-cache and no-store.

no-cache: response may be cached, but cache must revalidate with origin before serving it (using If-None-Match or If-Modified-Since). no-store: response must never be stored in any cache — not browser, not proxy. Use no-store only for truly sensitive data (authentication tokens, PII).

Q3. What does Cache-Control: must-revalidate do?

Once a cached response becomes stale (max-age exceeded), must-revalidate forbids serving the stale copy. Cache must revalidate with origin. Without must-revalidate, some caches may serve stale content under certain conditions (e.g., network partition). Essential for data where staleness is unacceptable.

Q4. What is an ETag?

Entity Tag: an opaque token (string) representing a specific version of a resource. Strong ETag: byte-identical match. Weak ETag (W/"..."): semantically equivalent. Server includes ETag in response; client sends it in If-None-Match on subsequent requests. Server returns 304 if resource hasn't changed.

Q5. What is the difference between ETag and Last-Modified?

Last-Modified is a date/time — used with If-Modified-Since. Imprecise (1-second granularity, clock skew). ETag is a content hash or version token — used with If-None-Match. More reliable. ETag is preferred. Both can be combined; server returns 304 if either condition passes.

Q6. How does Spring's ShallowEtagHeaderFilter work?

It wraps the response, buffers the entire response body, computes MD5 hash as ETag, and either returns the full response (first request) or 304 (if client's If-None-Match matches). Limitation: business logic always executes — only bandwidth is saved, not server CPU. Use deep ETags for CPU savings.

Q7. What is a deep ETag and how does it differ from shallow?

A deep ETag is based on a domain object's version/hash (e.g., `entity.getVersion()`), not the response body. `WebRequest.checkNotModified(etag)` returns true (304) before the service method body executes, skipping DB queries and computation entirely. Far more efficient than `ShallowEtagHeaderFilter`.

Q8. What is Cache-Control: private vs public?

public: response may be cached by any cache including shared caches (CDNs, proxies). private: response may only be cached by browser (client) cache — not CDN or proxy. Use private for authenticated user-specific responses. Use public for static assets and shared API responses.

Q9. What is s-maxage?

s-maxage overrides max-age for shared caches (CDNs, reverse proxies) only. Browser still uses max-age. Use `s-maxage=3600, max-age=60` to cache longer at CDN (1 hour) while browser cache expires quickly (1 min). Useful when you can do CDN purges but not browser cache invalidation.

Q10. What does Cache-Control: stale-while-revalidate do?

Allows serving a stale response immediately while asynchronously revalidating in background. Response: `Cache-Control: max-age=60, stale-while-revalidate=600`. For 60s it's fresh. 60-660s it's served stale while background refresh happens. After 660s it's fully stale. Eliminates cache-miss latency spikes for frequently accessed resources.

Q11. What is Cache-Control: immutable?

Tells browser not to revalidate the resource during its max-age period, even when the user explicitly refreshes. Ideal for content-hashed assets (`app-8f4d2a9c.js`) that never change for a given URL. Eliminates conditional GET requests for versioned static files.

Q12. What is the Vary header and why is it important?

Vary lists request headers that the response depends on. Cache creates separate entries per combination of Vary values. Vary: Accept-Encoding means gzip and identity compressed responses are cached separately. Vary: Accept handles JSON vs XML. Avoid Vary: Authorization — creates N entries per user, defeats sharing.

Q13. What is a conditional GET?

GET request with If-None-Match (ETag) or If-Modified-Since headers. Server returns 304 Not Modified (no body) if condition passes. Client reuses cached copy. Saves bandwidth even when max-age has expired. Always combine max-age (freshness) with ETags (validation) for best performance.

Q14. How do you implement HTTP caching in Spring Boot REST APIs?

Use `ResponseEntity.cacheControl(CacheControl.maxAge(...))` for simple caching. Use `WebRequest.checkNotModified(etag)` for conditional GET with version-based ETags. Register `ShallowEtagHeaderFilter` for content-based ETags. Configure resource handlers with `setCacheControl` for static assets.

Q15. What HTTP status code is returned for a successful cache validation?

304 Not Modified. No response body is transmitted — saves bandwidth. Client uses its cached copy with updated headers. Compare with 200 OK (fresh resource) and 412 Precondition Failed (for conditional writes with If-Match).

Q16. What is the role of the Age header?

Age: N header indicates how many seconds the response has been in a shared cache. Remaining freshness = max-age - Age. If Age > max-age, response is stale. CDNs add Age to responses. Useful for diagnosing stale content issues.

Q17. How do you prevent caching of dynamic personalized content?

Use Cache-Control: no-store for highly sensitive content. Use Cache-Control: no-cache, private for personalized but not sensitive content (revalidation still possible). Include Pragma: no-cache for HTTP/1.0 compatibility. Set Surrogate-Control: no-store for CDN-specific instructions.

Q18. What is a cache key?

The identifier used to store/retrieve a cached response. Default HTTP cache key: method + URL. With Vary headers: key = method + URL + specified request header values. In CDNs: configurable; often includes query string, cookies, custom headers. Normalize URLs (sort query params) to maximize cache hit rate.

Q19. What is stale-if-error?

Cache-Control: stale-if-error=86400 instructs caches to serve stale content if origin returns 5xx or is unreachable, for up to 86400 seconds. Provides resilience: users see slightly stale data rather than error pages during origin outages. Supported by CDNs like Cloudflare and Fastly.

Q20. How do ETags work in REST API collections?

Collection ETag can be a hash of the collection's version/fingerprint or a composite of all member ETags. On collection modification, ETag changes. Client's If-None-Match check prevents re-downloading unchanged collections. Implementation: use database version column or last-modified timestamp of the newest member.

Q21. What is content hashing / fingerprinting for static assets?

Include a hash of the file content in the URL (e.g., app-8f4d2a9c.js). When content changes, URL changes. Old URL can be cached indefinitely (Cache-Control: max-age=31536000, immutable). New URL is always fetched fresh. Spring's VersionResourceResolver adds content hash automatically.

Q22. What is the problem with Vary: * (star)?

Vary: * means every request is unique — cache must revalidate every time. Effectively disables caching. Avoid it. Use specific headers instead. CDNs typically treat Vary: * as uncacheable.

Q23. How do you handle cache invalidation for HTTP caches?

Proactive: CDN purge API (CloudFront invalidation, Cloudflare purge). Event-driven purge on content update. Surrogate-Key/Cache-Tag: tag cached responses, purge by tag. Versioned URLs: change URL when content changes. Short TTLs: eventual consistency via expiry. Note: browser caches cannot be purged directly.

Q24. What is Surrogate-Control vs Cache-Control for CDNs?

Cache-Control is the standard header visible to all caches including browsers. Surrogate-Control is a CDN-specific header (Varnish, Fastly) with longer TTLs. CDN strips Surrogate-Control before forwarding to client. Allows CDN to cache for hours while browser caches for minutes.

Q25. How does Spring Security affect HTTP caching?

Spring Security by default adds Cache-Control: no-cache, no-store, max-age=0, must-revalidate and Pragma: no-cache to all authenticated responses. To enable caching for secure resources, configure `HttpSecurity.headers().cacheControl().disable()` and set headers manually per endpoint.

Q26. What is the Cache-Control: only-if-cached directive?

Client request directive: client wants only cached responses — if no cached response available, return 504 Gateway Timeout rather than forwarding to origin. Used for offline-first apps and bandwidth-sensitive clients. Not commonly used in server-to-server API calls.

Q27. Explain the priority order of Cache-Control directives.

For shared caches: s-maxage > max-age > Expires. no-store takes precedence over everything (response never cached). no-cache overrides max-age (must revalidate). must-revalidate means no stale serving. private means shared caches must not store. Order of precedence ensures security-sensitive directives win.

Q28. What HTTP caching mechanism works best for API responses with authenticated users?

Cache-Control: no-cache, private with ETag. no-cache forces revalidation; private restricts to browser; ETag enables 304 conditional responses to save bandwidth. Authenticated responses must not be cached by shared caches (CDN/proxy) unless user-specific CDN caching with tokens is configured.

Q29. How do you implement URL-based cache versioning in Spring?

Spring `WebMvcConfigurer.addResourceHandlers()` with `VersionResourceResolver`. Two strategies: `ContentVersionStrategy` (MD5 hash of file content — URL changes when content changes), `FixedVersionStrategy` (explicit version string — e.g., app version). Thymeleaf/JSP use `@{/static/app.js}` to get version-resolved URL from `ResourceUrlProvider`.

Q30. What is the difference between browser cache, CDN cache, and reverse proxy cache?

Browser cache: private, per-user, on device; cannot be invalidated externally. CDN cache: shared, geo-distributed, edge nodes closest to users; support purge APIs. Reverse proxy cache (Nginx, Varnish): shared, in data center, reduces origin load; configurable. HTTP caching headers control all three; `Surrogate-Control/Cache-Tag` provide CDN-specific control.

Q31. What are common mistakes in HTTP caching configuration?

1) Caching POST responses (not idempotent — must not cache). 2) Vary: Authorization (prevents any CDN caching). 3) Not setting Cache-Control on static assets (defaults to no caching or inconsistent behavior). 4) Setting max-age=0 instead of no-store for sensitive data. 5) Not combining max-age with ETag — missing validation benefits when max-age expires. 6) Forgetting to update ETag/version when data changes (stale data served forever).

Q32. How does the Nginx proxy_cache_use_stale directive improve resilience?

`proxy_cache_use_stale error timeout updating http_500 http_502 http_503` instructs Nginx to serve stale cached content when the origin returns errors or is slow. 'updating' serves stale while one request refreshes the cache (SWR behavior). Improves fault tolerance: users see stale data, not errors.

Q33. What is the X-Cache header?

Non-standard but widely used header added by CDNs and proxies to indicate cache status: X-Cache: HIT (served from cache), MISS (fetched from origin), BYPASS (cache skipped), EXPIRED (stale). Nginx adds X-Cache-Status via `$upstream_cache_status`. Useful for diagnosing caching issues.